

Week 02: Problem Solving by Search: Routes, Plans and Paths

Artificial Intelligence Applications in Engineering (MUH-920 Mühendislikte Yapay Zeka Uygulamaları). Lecture notes

Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye
Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University
Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University

Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)

IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 | utkukose.com | github.com/utkukose

utkukose@sdu.edu.tr | utku.kose@und.edu | ukose@up.edu.mx | utkukose@gmail.com

Week overview, study path and tasks: <https://github.com/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-02/README.md>

Lecture page with animations and knowledge checks: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-02/lecture.html>

Interactive lab with self-assessment and reflection: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-02/lab.html>

Colab notebook with Python step 2 and the hands-on work:

https://colab.research.google.com/github/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-02/NB02_search_and_planning.ipynb

Content licensed under CC BY 4.0. This course is updated in line with current developments in the field.

Last update: October 2026.

Overview

Many engineering decisions are sequences: the route of a truck, the order of drilling holes, the moves of a crane or the path of a pipeline across a valley. Search turns such decisions into the exploration of a state space and finds a sequence of actions that reaches a goal at least cost [1, 2]. This week formulates problems as state spaces, compares uninformed search with heuristic search, proves why A* returns optimal paths when its heuristic never overestimates, and applies the ideas to path planning for vehicles on real terrain near Isparta [3, 4, 5].

Estimated study time: 8 to 10 hours.

Learning outcomes

By the end of the week, students are expected to formulate an engineering task as a search problem with states, actions, transition model, goal test and path cost, to trace breadth-first search, uniform-cost search and A* by hand, to judge whether a heuristic is admissible and consistent, to explain the effect of a heuristic on the number of expanded nodes, and to implement grid path planning with terrain costs in Python.

Python in this week

These notes close with Python step 2: Lists, tuples and loops. It covers lists, tuples, for and while loops, sets and a first queue, applied to routes and grid maps like those searched in this week [6]. In the course materials, the step is taught at the start of the Colab notebook, where each snippet runs in a cell next to

its explanation and short exercises give immediate feedback, and later sections of the notebook mark further constructs as Python moves.

Problems as state spaces

A search problem has five parts [1]. The initial state describes where the agent starts. The actions available in a state say what the agent can do. The transition model gives the state that results from an action. The goal test decides whether a state solves the problem, and the path cost adds up the cost of the actions along a sequence. A solution is a sequence of actions from the initial state to a goal state, and an optimal solution has the lowest path cost among all solutions.

The formulation is an engineering decision in its own right. For a haul truck in an open-pit mine, a state can be a node of the road network, and the path cost can be travel time, fuel or tyre wear. For a mobile robot in a factory, the state can be a cell of an occupancy grid, which divides the floor into squares that are free or blocked [4]. For a crane that stacks containers, the state is the arrangement of all containers, and the number of states grows combinatorially. A good formulation keeps only the details that matter for the decision: A route planner for a truck does not need the colour of the truck, but it may need the slope of every road segment.

Check your understanding. A planner chooses the order in which a laser cutter visits 60 contours on a steel sheet. What is a natural state in the formulation?

- A. The colour of the steel sheet
- B. The set of contours already cut together with the current position of the cutting head
- C. The electrical power of the laser only
- D. The final cutting plan

Answer: B. The future cost depends on which contours remain and where the head is, so both belong in the state.

Formulas: A search problem

A search problem is written with five parts [1]. The initial state is s_0 . The set of actions available in state s is $A(s)$. The transition model $T(s, a)$ returns the state reached by doing action a in state s . The goal test decides whether a state is a goal. The step cost $c(s, a, s')$ is the cost of moving from s to s' with action a . A solution is a sequence of actions $a_1, a_2, \dots, a_m$ that leads through the states $s_0, s_1, \dots, s_m$ to a goal state, and its path cost is the sum of the step costs:

$$g = \sum_{i=1}^m c(s_{i-1}, a_i, s_i)$$

An optimal solution has the smallest path cost C^* among all solutions. The size of the state space decides which methods are feasible. An occupancy grid with R rows and K columns has $R \times K$ cells. A robot that also has one of H headings has $R \times K \times H$ states. Orders of n jobs are counted by the factorial

$$n! = n \times (n-1) \times (n-2) \times \dots \times 2 \times 1$$

because the first job can be chosen in n ways, the second in $n-1$ ways, and so on. The factorial also follows step by step from $n! = n \times (n-1)!$ with $1! = 1$.

Worked example: How large are three state spaces?

Step 1. A factory floor. A grid with 40 rows and 25 columns has $40 \times 25 = 1000$ cells, so a search for a mobile robot has at most 1000 states.

Step 2. The same floor with headings. A vehicle that cannot turn on the spot must also remember one of 8 headings: $1000 \times 8 = 8000$ states.

Step 3. Orders of jobs. The factorial grows by one multiplication per job:

$$1! = 1, 2! = 2 \times 1 = 2, 3! = 3 \times 2 = 6, 4! = 4 \times 6 = 24, 5! = 5 \times 24 = 120$$

$$6! = 6 \times 120 = 720, 7! = 7 \times 720 = 5040, 8! = 8 \times 5040 = 40320$$

$$9! = 9 \times 40320 = 362880, 10! = 10 \times 362880 = 3628800$$

Continuing in the same way gives $15! = 1307674368000$ and $20! = 2432902008176640000$, about 2.43×10^{18} , the “more than two quintillion” orders of the lecture.

Step 4. Time to try them all. A computer that evaluates 10^9 orders per second needs

$$\frac{2.43 \times 10^{18}}{10^9} = 2.43 \times 10^9 \text{ s}, \frac{2.43 \times 10^9}{3.156 \times 10^7} = 77 \text{ years}$$

since a year has about 3.156×10^7 seconds. Enumeration is hopeless already for twenty jobs, while the grid of Step 1 is searched in a fraction of a second.

Uninformed search: Breadth first, depth first and uniform cost

Uninformed strategies know nothing about the goal beyond the goal test. They differ only in the order in which they expand nodes of the search tree. Breadth-first search expands the shallowest node first by keeping the frontier in a first-in first-out queue. It finds the solution with the fewest actions, which is optimal when every action has the same cost. Depth-first search expands the deepest node first with a last-in first-out stack, needs little memory, but may wander along an infinite branch and does not guarantee the shortest solution. Iterative deepening repeats depth-limited search with growing limits and combines the memory use of depth-first search with the completeness of breadth-first search [7].

When actions have different costs, the fewest actions is not the cheapest path. Uniform-cost search expands the node with the lowest path cost so far, which is Dijkstra’s algorithm on an explicit graph [8]. It is optimal for non-negative costs, because a node is expanded only when no cheaper route to it can exist.

The price of uninformed search is growth. With branching factor b and solution depth d , breadth-first search may generate on the order of b to the power d nodes. On a grid with four moves and a goal forty steps away, blind search explores almost every reachable cell within that radius. The number becomes unmanageable for combinatorial problems such as sequencing: Twenty jobs can be ordered in more than two quintillion ways.

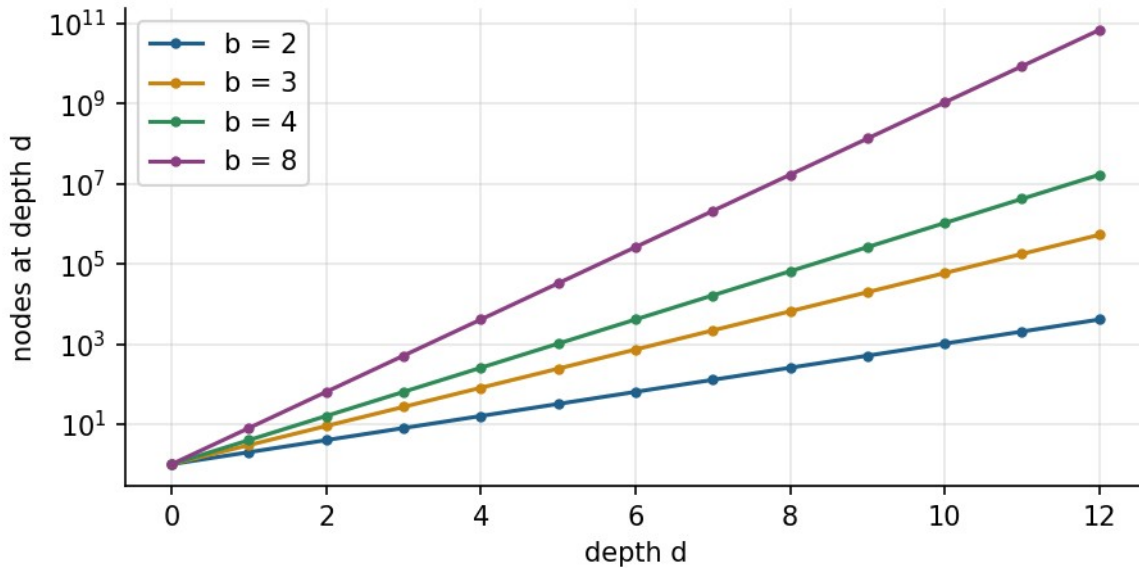


Figure 2.1. Growth of the number of nodes in a complete search tree with branching factor b . Uninformed search must be guided or limited when b or d is large [1].

Check your understanding. Road segments of a network have different travel times. Which strategy returns the fastest route without extra knowledge about the goal?

- A. Breadth-first search
- B. Depth-first search
- C. Uniform-cost search
- D. Random walk

Answer: C. Uniform-cost search, Dijkstra's algorithm, expands the node with the lowest accumulated cost, so it is optimal for non-negative costs.

Formulas: Search trees, frontiers and uniform cost

Search builds a tree whose root is the initial state. If every state has b successors, the branching factor, then depth k of the tree holds b^k nodes, and a complete tree of depth d holds

$$N(b, d) = 1 + b + b^2 + \dots + b^d = \sum_{k=0}^d b^k = \frac{b^{d+1} - 1}{b - 1}$$

nodes, by the formula for a geometric series. Breadth-first search must keep the whole deepest level in memory, about b^d nodes, while depth-first search keeps one path with its unexplored siblings, about $b \times d$ nodes. Iterative deepening runs depth-limited searches with limits $0, 1, \dots, d$, so a node at depth k is generated $d + 1 - k$ times:

$$N_{IDS}(b, d) = \sum_{k=0}^d (d + 1 - k) b^k$$

Uniform-cost search stores for every reached state the cost $g(n)$ of the cheapest path found so far. When node n is expanded, each neighbour n' receives the candidate cost $g(n) + c(n, n')$, and the stored value is replaced only if the candidate is smaller:

$$g(n') \leftarrow \min(g(n'), g(n) + c(n, n'))$$

The arrow means “is replaced by”. The frontier node with the smallest g is expanded next, which is Dijkstra's algorithm [8]. Because costs are not negative, a node's g can no longer improve once it is expanded, and the cost of the goal at expansion is optimal.

Worked example: How fast does the tree grow?

A grid robot has $b = 4$ moves, and the goal lies $d = 5$ moves away.

Step 1. Nodes per level. $4^0 = 1$, $4^1 = 4$, $4^2 = 16$, $4^3 = 64$, $4^4 = 256$ and $4^5 = 1024$.

Step 2. The whole tree, by adding the levels.

$$N(4, 5) = 1 + 4 + 16 + 64 + 256 + 1024 = 1365$$

Step 3. The same number from the closed formula.

$$N(4, 5) = \frac{4^6 - 1}{4 - 1} = \frac{4096 - 1}{3} = \frac{4095}{3} = 1365$$

Step 4. Iterative deepening. The root is generated six times, the nodes of depth 1 five times, and so on:

$$N_{\text{IDS}}(4, 5) = 6 \times 1 + 5 \times 4 + 4 \times 16 + 3 \times 64 + 2 \times 256 + 1 \times 1024 = 6 + 20 + 64 + 192 + 512 + 1024 = 1818$$

Iterative deepening generates $1818/1365 = 1.332$ times as many nodes as one breadth-first pass, only about a third more, but it needs memory for one path only: $b \times d = 4 \times 5 = 20$ nodes instead of the $4^5 = 1024$ nodes of the deepest level. Part E of the [interactive lab](#) repeats the count for any branching factor and depth.

Worked example: Breadth-first and uniform-cost search on a road network

The examples of this week use a small road network with seven junctions. Each junction has map coordinates in kilometres, and each road has a travel time in minutes. Every travel time is at least as large as the straight-line distance between its two junctions, as on real roads. Part C of the [interactive lab](#) draws the network and runs every algorithm on it step by step.

Junction	Coordinates (x, y) in km
S	$(0, 0)$
A	$(2, 3)$
B	$(3, 0)$
C	$(5, 3)$
D	$(6, 0)$
E	$(4, -2)$
G	$(8, 1)$

Road	Travel time c (min)
S to A	4
S to B	3
S to E	6
A to B	4
A to C	3
B to C	5
B to D	4
C to G	4
D to G	3

Road	Travel time c (min)
E to G	6

Breadth-first search. The queue starts with S. Neighbours are added in alphabetical order, and a junction that has been reached before is not added again.

Expansion 1. Expand S; the new neighbours A, B, E join the back of the queue. Queue: A, B, E.

Expansion 2. Expand A; the new neighbours C join the back of the queue. Queue: B, E, C.

Expansion 3. Expand B; the new neighbours D join the back of the queue. Queue: E, C, D.

Expansion 4. Expand E; the new neighbours G join the back of the queue. Queue: C, D, G.

Expansion 5. Expand C; all neighbours have been reached before. Queue: D, G.

Expansion 6. Expand D; all neighbours have been reached before. Queue: G.

Expansion 7. G is taken from the front of the queue and passes the goal test.

Breadth-first search returns the route S to E to G with 2 roads, the fewest possible, but its travel time is $6+6=12$ minutes.

Uniform-cost search. The frontier starts with $g(S)=0$. In each expansion the junction with the smallest g leaves the frontier; ties are broken alphabetically. The frontier lists junctions in the order in which they would be expanded.

Expansion 1. Expand S with $g(S)=0$: $g(A)=0+4=4$ (new); $g(B)=0+3=3$ (new); $g(E)=0+6=6$ (new). Frontier: B, A, E.

Expansion 2. Expand B with $g(B)=3$: A would get $3+4=7$, which is not better than 4, so nothing changes; $g(C)=3+5=8$ (new); $g(D)=3+4=7$ (new); S is already expanded. Frontier: A, E, D, C.

Expansion 3. Expand A with $g(A)=4$: B is already expanded; C gets $4+3=7$, which is better than 8, so $g(C)=7$; S is already expanded. Frontier: E, C, D.

Expansion 4. Expand E with $g(E)=6$: $g(G)=6+6=12$ (new); S is already expanded. Frontier: C, D, G.

Expansion 5. Expand C with $g(C)=7$: A is already expanded; B is already expanded; G gets $7+4=11$, which is better than 12, so $g(G)=11$. Frontier: D, G.

Expansion 6. Expand D with $g(D)=7$: B is already expanded; G gets $7+3=10$, which is better than 11, so $g(G)=10$. Frontier: G.

Expansion 7. G has the smallest cost on the frontier, $g(G)=10$, and passes the goal test.

Following the stored predecessors back from G gives the route S to B to D to G with travel time $3+4+3=10$ minutes, the optimum. Uniform-cost search expanded 7 junctions to prove it.

Informed search: Heuristics and A*

A heuristic function $h(n)$ estimates the cost from node n to the nearest goal. Greedy best-first search expands the node with the smallest h and often reaches a goal quickly, but it can be misled into long detours. A* combines the cost already paid, $g(n)$, with the estimate, and expands the node with the smallest $f(n) = g(n) + h(n)$ [3].

Hart, Nilsson and Raphael showed that A* returns an optimal path if the heuristic is admissible, which means it never overestimates the true remaining cost [3]. The straight-line distance is admissible for road travel, because no road is shorter than a straight line. On a grid with four moves, the Manhattan

distance, the sum of horizontal and vertical offsets, is admissible. A heuristic is consistent when its estimate never drops by more than the cost of one step. Consistency implies admissibility and guarantees that A* never needs to reopen a node [2].

Better heuristics save work. If one admissible heuristic is always at least as large as another, A* with the larger one expands no more nodes. The zero heuristic turns A* into uniform-cost search, and a perfect heuristic walks straight to the goal. Weighted A* multiplies the heuristic by a factor larger than one: It usually expands far fewer nodes and returns a path whose cost is at most that factor times the optimum, which is a common engineering compromise when planning must meet a deadline.

Animation: Breadth-first search and A* on the same map. Both searches start at the green cell and look for the red cell. Blue cells have been expanded. Compare how many cells each algorithm expands before it finds a path of the same length. Run it on the [lecture page](#).

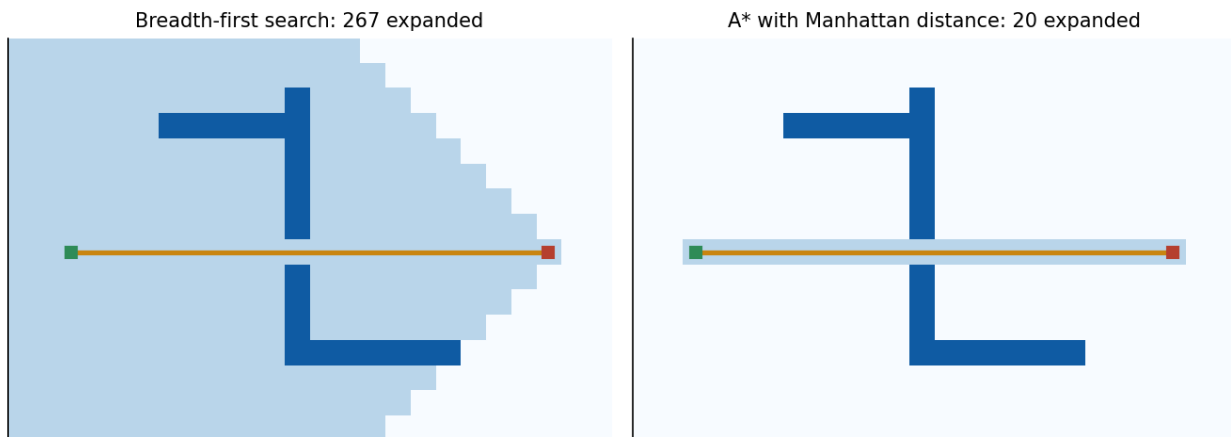


Figure 2.2. Expanded cells of breadth-first search and A* with the Manhattan heuristic on the same grid. Both return a shortest path, but A* expands far fewer cells [3].

Check your understanding. Which heuristic is admissible for a robot that moves on a grid with four moves of cost 1?

- A. Twice the Manhattan distance
- B. The Manhattan distance
- C. The number of obstacles on the map
- D. The square of the Euclidean distance

Answer: B. The Manhattan distance is the length of a path without obstacles, so it can never exceed the true remaining cost.

Formulas: Heuristics and A*

A heuristic $h(n)$ estimates the cost from node n to the nearest goal, and $g(n)$ is the cost already paid to reach n . Greedy best-first search expands the node with the smallest $h(n)$. A* expands the node with the smallest evaluation [3]:

$$f(n) = g(n) + h(n)$$

Let $h^*(n)$ be the true cost of the cheapest path from n to a goal. A heuristic is admissible when it never overestimates:

$$h(n) \leq h^*(n) \text{ for every node } n$$

and consistent when the estimate never falls by more than the cost of one step from n to a neighbour n' :

$$h(n) \leq c(n, n') + h(n')$$

Two common heuristics measure the offsets Δx and Δy between a node and the goal. The Manhattan distance suits grids with four moves, and the straight-line, or Euclidean, distance suits roads and free movement [2]:

$$h_M = |\Delta x| + |\Delta y|, h_E = \sqrt{\Delta x^2 + \Delta y^2}$$

The zero heuristic $h=0$ turns A* into uniform-cost search. Weighted A* expands by $f_w(n) = g(n) + w h(n)$ with a weight $w > 1$; with an admissible h , the cost C of its solution satisfies $C \leq w C^*$.

Worked example: A* on the same road network

Step 1. Heuristic values. The goal G lies at (8, 1). The straight-line distance from each junction, rounded to three decimal places, is

Junction	Calculation	$h(n)$
S	$\sqrt{(8-0)^2 + (1-0)^2} = \sqrt{64+1} = \sqrt{65}$	8.062
A	$\sqrt{(8-2)^2 + (1-3)^2} = \sqrt{36+4} = \sqrt{40}$	6.325
B	$\sqrt{(8-3)^2 + (1-0)^2} = \sqrt{25+1} = \sqrt{26}$	5.099
C	$\sqrt{(8-5)^2 + (1-3)^2} = \sqrt{9+4} = \sqrt{13}$	3.606
D	$\sqrt{(8-6)^2 + (1-0)^2} = \sqrt{4+1} = \sqrt{5}$	2.236
E	$\sqrt{(8-4)^2 + (1-(-2))^2} = \sqrt{16+9} = \sqrt{25}$	5
G	$\sqrt{(8-8)^2 + (1-1)^2} = \sqrt{0+0} = \sqrt{0}$	0

Step 2. Admissibility. The cheapest travel times to G, found by uniform-cost search from every junction, are the values $h^*(n)$. Every estimate is smaller, as it must be, because no road is shorter than the straight line:

Junction	$h(n)$	$h^*(n)$	$h(n) \leq h^*(n)$
S	8.062	10	yes
A	6.325	7	yes
B	5.099	7	yes
C	3.606	4	yes
D	2.236	3	yes
E	5	6	yes
G	0	0	yes

The heuristic is also consistent. For the road from S to B, for example, $h(S) = 8.062 \leq c(S, B) + h(B) = 3 + 5.099 = 8.099$.

Step 3. The search. The frontier starts with S, and $f(S) = 0 + 8.062 = 8.062$.

Expansion 1. Expand S with $f(S) = 0 + 8.062 = 8.062$: $g(A) = 0 + 4 = 4$ and $f(A) = 4 + 6.325 = 10.325$; $g(B) = 0 + 3 = 3$ and $f(B) = 3 + 5.099 = 8.099$; $g(E) = 0 + 6 = 6$ and $f(E) = 6 + 5 = 11$. Frontier in order of f : B, A, E.

Expansion 2. Expand B with $f(B) = 3 + 5.099 = 8.099$: A would get $3 + 4 = 7$, which is not better than 4, so nothing changes; $g(C) = 3 + 5 = 8$ and $f(C) = 8 + 3.606 = 11.606$; $g(D) = 3 + 4 = 7$ and $f(D) = 7 + 2.236 = 9.236$; S is already expanded. Frontier in order of f : D, A, E, C.

Expansion 3. Expand D with $f(D) = 7 + 2.236 = 9.236$: B is already expanded; $g(G) = 7 + 3 = 10$ and $f(G) = 10 + 0 = 10$. Frontier in order of f : G, A, E, C.

Expansion 4. G has the smallest f on the frontier, $f(G) = 10 + 0 = 10$, and passes the goal test.

Step 4. Result. A* returns the same optimal route, S to B to D to G with 10 minutes, after 4 expansions instead of 7. Junctions A, C and E, which lie away from the goal or behind expensive roads, were never expanded, because their f values stayed above 10.

Path planning for vehicles and machines

Path planning in engineering adds physical constraints to graph search. Occupancy grids and road graphs give the search space, while costs encode travel time, energy, risk or wear [4]. Vehicles cannot turn on the spot, so planners for cars and trucks search over positions and headings. Hybrid A* keeps continuous positions inside discrete cells and has been used to park and manoeuvre autonomous vehicles in unstructured environments such as parking lots [9]. Surveys of self-driving urban vehicles place such graph-search planners next to sampling-based and optimisation-based methods [10].

Terrain adds cost and feasibility. A haul road in a mine, a pipeline or a forest road must respect a maximum grade, and steep segments cost more fuel and time. A digital elevation model turns the terrain into a grid of heights, and the cost of moving between two cells can combine distance with a penalty on slope and a hard limit above which the move is forbidden. The notebook applies this idea to a real elevation grid near Isparta from the Copernicus GLO-90 model, served by the Open-Meteo Elevation API [5, 11].

Formulas: Cost of a move across terrain

A digital elevation model stores the height z of every cell of a grid with spacing ℓ (ell), 90 metres for the GLO-90 model [5]. A move to one of the eight neighbouring cells, Δr rows and Δk columns away, has the horizontal length

$$d = \ell \sqrt{\Delta r^2 + \Delta k^2}$$

which is ℓ for a straight move and $\ell\sqrt{2}$ for a diagonal one. The grade of the move is the rise over the run, often quoted as a percentage:

$$\text{grade} = \frac{|z' - z|}{d}$$

The planner of the notebook forbids moves whose grade exceeds a limit $G_{\max}$ and charges the others with a slope penalty p :

$$c = d(1 + p \times \text{grade}) = d + p|z' - z|$$

The second form follows by multiplying out, since $d \times p \times |z' - z|/d = p|z' - z|$: Every metre of height difference costs p metres of extra distance. The heuristic $h = \ell \sqrt{\Delta r^2 + \Delta k^2}$ to the goal stays admissible, because no move costs less than its horizontal length.

Worked example: Three moves from one cell

A cell at $z = 1210$ m has three candidate neighbours. The planner uses $\ell = 90$ m, $p = 10$ and $G_{\max} = 0.25$.

Step 1. East, to $z' = 1225$ m. The length is $d = 90 \times \sqrt{0^2 + 1^2} = 90$ m, the rise is $|1225 - 1210| = 15$ m and the grade is $15/90 = 0.1667$, below the limit 0.25. The cost is

$$c = 90 \times (1 + 10 \times 0.1667) = 90 \times 2.667 = 240.03$$

or exactly $90 + 10 \times 15 = 240$ m with the second form; the small difference comes from rounding the grade.

Step 2. North-east, to $z' = 1240$ m. The diagonal length is

$d = 90 \times \sqrt{1^2 + 1^2} = 90 \times 1.414214 = 127.279$ m, the rise is 30 m and the grade is $30/127.279 = 0.2357$, still below 0.25. The cost is

$$c = 127.279 + 10 \times 30 = 127.279 + 300 = 427.279$$

Step 3. North, to $z' = 1235$ m. The grade is $25/90 = 0.2778$, which exceeds 0.25, so the move is forbidden and never enters the frontier.

The penalty makes the planner prefer the long way round a hill to a short climb over it: The diagonal move climbs 30 m and costs as much as a level route of 427 m.

Beyond systematic search: Local search

Systematic search remembers the paths it explores. Local search keeps only a current solution and improves it by small changes, which suits problems where the path does not matter and only the final configuration counts, such as a layout or a schedule. Hill climbing accepts only improvements and stops at the first local optimum. Simulated annealing sometimes accepts worse solutions, with a probability that falls as a temperature parameter is lowered, and can therefore escape local optima [12]. Weeks 5 and 6 develop this line into evolutionary and swarm algorithms, which keep a population of candidate solutions instead of one.

Check your understanding. Why can simulated annealing escape a local optimum where hill climbing stops?

- A. It evaluates every possible solution
- B. It sometimes accepts a worse solution, with a probability that decreases over time
- C. It uses the gradient of the objective
- D. It always restarts from the best known solution

Answer: B. Occasional uphill moves in cost let the search leave the basin of a local optimum early in the run.

Formulas: Hill climbing and simulated annealing

Local search keeps one current solution x with cost $f(x)$ and proposes a neighbour x' . The change in cost is

$$\Delta = f(x') - f(x)$$

Hill climbing, written here for minimisation, accepts the neighbour only if $\Delta < 0$. Simulated annealing always accepts improvements and accepts a worse neighbour, $\Delta > 0$, with the probability [12]

$$P(\text{accept}) = e^{-\Delta/T}$$

where the temperature $T > 0$ controls how tolerant the search is. In practice a random number u between 0 and 1 is drawn, and the worse neighbour is accepted if $u < e^{-\Delta/T}$. The temperature is lowered by a cooling schedule, most often geometric with a factor α (alpha) slightly below one:

$$T_k = T_0 \alpha^k$$

Solving $T_0 \alpha^k = T_{\text{end}}$ for k with logarithms gives the number of steps until the temperature reaches T_{end} :

$$k = \frac{\ln(T_{\text{end}}/T_0)}{\ln \alpha}$$

Worked example: Accepting a worse layout

A layout has cost $f(x) = 52$, and the proposed neighbour has $f(x') = 55$.

Step 1. Change in cost. $\Delta = 55 - 52 = 3$, a worsening. Hill climbing rejects the move.

Step 2. Acceptance probability at three temperatures.

$$T = 10: e^{-3/10} = e^{-0.3} = 0.740818$$

$$T = 2: e^{-3/2} = e^{-1.5} = 0.22313$$

$$T = 0.5: e^{-3/0.5} = e^{-6} = 0.002479$$

Step 3. Decision with a random number. The random number generator returns $u = 0.41$. At $T = 10$, $0.41 < 0.740818$, so the worse layout is accepted. At $T = 2$, $0.41 > 0.22313$, so it is rejected. Early in the search, when the temperature is high, the search moves freely between valleys of the cost landscape; late in the search it behaves almost like hill climbing.

Step 4. Cooling schedule. With $T_0 = 10$ and $\alpha = 0.95$, the temperature after 20 steps is

$$T_{20} = 10 \times 0.95^{20} = 10 \times 0.358486 = 3.58486$$

and the number of steps until T falls to 0.1 is

$$k = \frac{\ln(0.1/10)}{\ln 0.95} = \frac{-4.60517}{-0.051293} = 89.78$$

so the temperature drops below 0.1 after 90 steps. Part D of the [interactive lab](#) runs simulated annealing on a cost profile with these formulas and shows each decision.

Python step 2: Lists, tuples and loops

Lists

A list holds several values in a fixed order and is written in square brackets. Positions are counted from 0, so `heights_m[0]` is the first element, and negative positions count from the end. A slice such as `heights_m[2:5]` returns the elements at positions 2, 3 and 4: The start is included and the stop is excluded. Lists can grow with `append`, and `len`, `min`, `max` and `sum` work on any list of numbers. The list below holds the ground elevation of successive cells along a planned route.

```
heights_m = [1012, 1018, 1031, 1047, 1040, 1066, 1080]
print(len(heights_m), heights_m[0], heights_m[-1])
print(heights_m[2:5])
heights_m.append(1093)
print(max(heights_m) - min(heights_m), "m between the lowest and the highest cell")
```

Output

```
7 1012 1080
[1031, 1047, 1040]
81 m between the lowest and the highest cell
```

Loops over lists

A `for` loop can run over the elements of a list directly or over positions produced by `range`. The loop below adds up all rises between neighbouring cells, which gives the total climb that a vehicle must make. The operator `+=` adds to a variable in place. The function `enumerate` supplies position and value together, which is often clearer than indexing by hand.

```
total_climb = 0
for i in range(1, len(heights_m)):
    rise = heights_m[i] - heights_m[i - 1]
    if rise > 0:
        total_climb += rise
print("total climb:", total_climb, "m")
for i, h in enumerate(heights_m[:3]):
    print("cell", i, "is at", h, "m")
```

Output

```
total climb: 88 m
cell 0 is at 1012 m
cell 1 is at 1018 m
cell 2 is at 1031 m
```

Grids and tuples

A map for path planning can be written as a list of strings, one string per row, with `#` marking obstacles. `grid[r][c]` then reads the character in row `r` and column `c`. A cell is naturally described by a tuple (row, column). Tuples are written with parentheses and, unlike lists, cannot be changed after creation, which suits fixed coordinates. Two nested loops visit every cell, and unpacking assigns the parts of a tuple to separate names.

```
grid = ["S..#....",
        ".#.#.##.",
        ".#...#...",
        "...#...G"]
rows, cols = len(grid), len(grid[0])
for r in range(rows):
    for c in range(cols):
        if grid[r][c] == "S":
            start = (r, c)
        elif grid[r][c] == "G":
            goal = (r, c)
print("start", start, "goal", goal)
r, c = goal
print("the goal is in row", r, "and column", c)
```

Output

```
start (0, 0) goal (3, 7)
the goal is in row 3 and column 7
```

The neighbours of a cell follow from four moves. A move is valid when it stays inside the map and does not enter an obstacle, and every valid neighbour is appended to a list.

```
moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]    # up, down, left, right
cell = (1, 2)
free_neighbours = []
for dr, dc in moves:
    nr, nc = cell[0] + dr, cell[1] + dc
```

```

    if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] != "#":
        free_neighbours.append((nr, nc))
print(free_neighbours)

```

Output

```
[(0, 2), (2, 2)]
```

While loops, sets and a queue

A while loop repeats as long as its condition holds, which suits searches whose length is not known in advance. Breadth-first search keeps a queue of cells to explore: New cells join at the back, and the next cell to expand leaves from the front. `collections.deque` provides such a queue with `append` and `popleft`. A set, written with braces, stores the cells already seen and answers membership questions such as `nxt not in visited` quickly. A dictionary `parent` records from which cell each cell was reached; Week 3 explains dictionaries in detail. The statement `break` leaves a loop early.

```

from collections import deque

frontier = deque([start])
visited = {start}
parent = {start: None}
while frontier:                                # an empty queue counts as False
    cell = frontier.popleft()
    if cell == goal:
        break
    for dr, dc in moves:
        nxt = (cell[0] + dr, cell[1] + dc)
        inside = 0 <= nxt[0] < rows and 0 <= nxt[1] < cols
        if inside and grid[nxt[0]][nxt[1]] != "#" and nxt not in
visited:
            visited.add(nxt)
            parent[nxt] = cell
            frontier.append(nxt)

path = []
while cell is not None:
    path.append(cell)
    cell = parent[cell]
path.reverse()
print(len(path) - 1, "moves:", path)

```

Output

```
10 moves: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (3,
4), (3, 5), (3, 6), (3, 7)]
```

The second while loop walks back from the goal through the recorded parents until it reaches the start, whose parent is `None`, and `reverse` puts the path in travel order. The notebook of this week extends the same pattern with a priority queue for A*.

Check your understanding. After `heights = [5, 7, 9, 11]`, what does `print(heights[1:3], heights[-1])` display?

- A. [7, 9] 11
- B. [5, 7, 9] 11
- C. [7, 9, 11] 5
- D. [7, 9] 9

Answer: A. The slice includes position 1 and stops before position 3, and position -1 is the last element.

Discipline challenges

Each row names a search problem from one department. Write its formulation and propose a heuristic, then decide whether systematic search, weighted search or local search suits its size.

Department	Challenge
Mining Engineering	Haul-road routing between a loading point and the crusher with a maximum grade and a fuel cost per metre of climb.
Civil Engineering	Corridor selection for a road or pipeline across a digital elevation model with slope and land-use costs.
Industrial Engineering	Order of picking items in a warehouse aisle system to minimise walking distance.
Electrical and Electronics Engineering	Routing a cable harness or a printed-circuit trace around keep-out zones.
Mechanical Engineering	Collision-free path of a robot arm tool point between two fixtures.
Automotive Engineering	Parking manoeuvre planning with position and heading states, as in hybrid A*.
Computer Engineering	Shortest route of packets in a network with link delays as costs.
Geological Engineering and Earth Sciences Engineering	Field survey route that visits outcrops on steep terrain with limited walking time.
Chemistry and Chemical Engineering	Reaction route from a starting compound to a target through known transformations with yields as costs.
Mathematics	Proof that consistency implies admissibility, and a counterexample in which an inadmissible heuristic misleads A*.

Weekly task

Formulate one search problem from your department with states, actions, transition model, goal test and path cost, and justify an admissible heuristic for it in about 400 words. Then run the terrain planner of the notebook with two different slope penalties and report how path length, total climb and expanded cells change. Cite at least three works from this week's references [1, 3, 4].

Research and report assignment (optional)

Path planning in practice. Review how path planning is done in one application area: autonomous haul trucks in mining, pipeline or road route selection on digital elevation models, or motion planning for automated vehicles. Compare graph-search planners with sampling-based or optimisation-based planners and discuss the constraints of the vehicle and the terrain [4, 9, 10].

Both assignments are optional. During an active semester, the weekly task can be sent together with the learning log exported from the interactive lab, and the research report on its own, to utkukose@sdu.edu.tr or utkukose@gmail.com. They carry no separate weight, but the instructor may take them into account as a discretionary adjustment of the midterm component of the grade. The assessment section of the syllabus gives the details, and research reports follow the report template of the course repository.

References

- [1] Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- [2] Pearl, J. (1984). *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley.
- [3] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100-107. <https://doi.org/10.1109/TSSC.1968.300136>
- [4] LaValle, S. M. (2006). *Planning Algorithms*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511546877>
- [5] European Space Agency (2021). *Copernicus Global Digital Elevation Model (GLO-90)*. <https://doi.org/10.5270/ESA-c5d3d65>
- [6] Python Software Foundation (2026). *The Python Tutorial (Python 3 documentation)*. <https://docs.python.org/3/tutorial/>
- [7] Korf, R. E. (1985). Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence*, 27(1), 97-109. [https://doi.org/10.1016/0004-3702\(85\)90084-0](https://doi.org/10.1016/0004-3702(85)90084-0)
- [8] Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1, 269-271. <https://doi.org/10.1007/BF01386390>
- [9] Dolgov, D., Thrun, S., Montemerlo, M., & Diebel, J. (2010). Path planning for autonomous vehicles in unknown semi-structured environments. *The International Journal of Robotics Research*, 29(5), 485-501. <https://doi.org/10.1177/0278364909359210>
- [10] Paden, B., Čáp, M., Yong, S. Z., Yershov, D., & Frazzoli, E. (2016). A survey of motion planning and control techniques for self-driving urban vehicles. *IEEE Transactions on Intelligent Vehicles*, 1(1), 33-55. <https://doi.org/10.1109/TIV.2016.2578706>
- [11] Open-Meteo (2026). *Open-Meteo free weather API: Historical Weather API and Elevation API (data licensed under CC BY 4.0)*. <https://open-meteo.com>
- [12] Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671-680. <https://doi.org/10.1126/science.220.4598.671>